

M1 변환 핵심 설계

2026. 10. 09.

목차

1. 요약
2. 배경과 목표
3. 설계
4. 구현 계획
5. 평가 방법
6. 위험과 대응

참고 출처

문서의 목적. M1에서 만들 변환 함수 `convert(value, src, dst)`의 데이터 구조, 계산 경로, 오류 규칙, 시험 방법을 설명하고, 구현에 들어가기 전에 방식과 완료 기준의 승인을 받는다.

1 요약

M1은 명령줄 없이 Python에서 부를 수 있는 변환 함수 하나와 그 시험을 만든다. M2는 이 함수를 감싸기만 한다[D1].

- **구조.** `convert.py`에 길이·무게 배수 사전 두 개, 온도 변환 식 사전 하나, 함수 `convert()` 하나를 둔다. `import`는 없다.
- **규칙.** 단위 이름은 대소문자를 가리지 않는다. 없는 단위, 종류가 다른 단위, 0 K 미만 온도는 `ValueError`를 낸다.
- **시험.** `test_convert.py(unittest)`에서 정의값 9개, 온도 기준점 5개, 같은 종류 단위 쌍 109개의 왕복, 오류 3종을 확인한다. 비교는 상대 오차 $1e-12$ 이내다.

2 배경과 목표

프로포절이 2026. 10. 09.에 승인되었다[D1]. 저장소에는 아직 코드가 없다[D2]. M1이 맡는 것은 프로포절 성공 기준(표 4) 중 정확성, 왕복 일관성, 의존성, 그리고 오류 처리 4종 중 3종이다. 남은 한 종(숫자가 아닌 값)은 문자열을 숫자로 바꾸는 단계에서 생기므로 M2가 맡는다. 표 1은 이 나눔이다.

표 1. 프로포절 성공 기준과 M1의 분담. [D1]

프로포절 기준	M1	M2
정확성	정의값·기준점 시험	—
왕복 일관성	모든 단위 쌍 시험	—
오류 처리	없는 단위, 종류 불일치, 0 K 미만	숫자 아닌 값, 종료 코드와 메시지
의존성	convert.py에 import 없음	argparse, sys만 추가

3 설계

그림 1은 convert() 안의 흐름이다. 단위 이름을 찾고, 두 단위가 같은 종류인지 확인한 뒤, 종류에 따라 두 경로 중 하나로 계산한다. 아래 점선은 오류로 빠지는 지점이다.

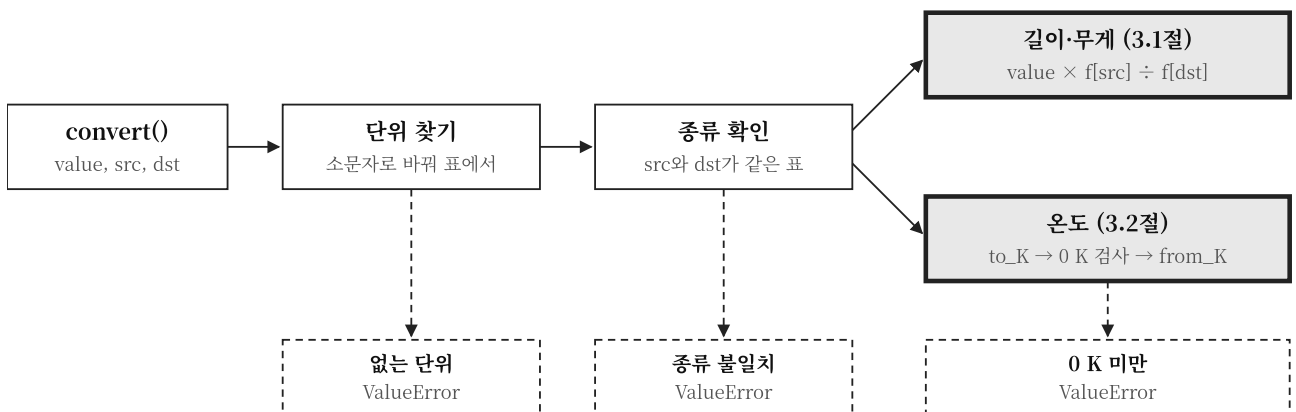


그림 1. convert() 안의 흐름. 굵은 테두리는 실제 계산이 일어나는 두 경로이고, 점선은 오류로 빠지는 길이다. 모두 이번에 새로 만드는 부분이다.

3.1 길이·무게: 배수 표

단위마다 기준 단위 하나에 해당하는 값(배수)을 사전에 적는다. 변환은 **원래 단위의 배수를 곱해 기준 단위로 만들고, 바꿀 단위의 배수로 나누는 것**이 전부다. 예를 들어 100 km → mi는 $100 \times 1000 \div 1609.344 = 62.137\cdots$ 이다. 배수는 프로포절 표 3의 정의값을 그대로 적는다[D1, R1, R2]. 표 2는 데이터 구조 세 개다.

표 2. `convert.py`의 데이터 구조. `f`는 기준 단위 하나에 해당하는 값이다. [D1, R1, R2]

이름	형태	내용
LENGTH	단위 → f (m)	mm 0.001, cm 0.01, m 1, km 1000, in 0.0254, ft 0.3048, yd 0.9144, mi 1609.344
MASS	단위 → f (kg)	mg 0.000001, g 0.001, kg 1, t 1000, lb 0.45359237, oz 0.45359237 / 16
TEMP	단위 → (켈빈으로, 켈빈에서)	c: (t + 273.15, k - 273.15), f: ((t + 459.67) ÷ 1.8, k × 1.8 - 459.67), k: (그대로, 그대로)

3.2 온도: 켈빈을 거치는 식

온도는 곱셈만으로 바꿀 수 없으므로 단위마다 “켈빈으로 바꾸는 식”과 “켈빈에서 바꾸는 식” 한 쌍을 둔다. 변환은 원래 단위의 앞 식으로 켈빈을 구하고, 바꿀 단위의 뒤 식을 적용한다. 단위가 n 개면 식 쌍도 n 개라 $^{\circ}\text{C} \leftrightarrow ^{\circ}\text{F}$ 같은 쌍별 식은 따로 적지 않는다. 식의 상수(32, 1.8, 459.67, 273.15)는 NIST 값이다[R1]. 켈빈 값이 0보다 작으면 물리적으로 없는 온도이므로 오류를 낸다. 0 K 자체(-273.15°C , -459.67°F)는 허용한다.

3.3 단위 이름 규칙

입력한 단위 이름은 소문자로 바뀌서 찾으므로 `KM`, `Km`, `km`가 모두 같다. 표의 키 17개를 소문자로 바꿔도 겹치는 것이 없음을 확인했다(표 2). 단위 이름은 기호만 받는다. `meter`, `celsius` 같은 긴 이름은 이번에 받지 않는다. `t`는 미터법 톤(1000 kg)이다.

3.4 오류

오류는 모두 `ValueError` 하나로 내고, 메시지에 이유를 적는다. 표 3이 세 경우와 메시지 예다. M2는 이 예외를 잡아 메시지를 출력하고 종료 코드 2로 끝내기만 하면 된다. 같은 단위끼리의 변환(`km → km`)은 오류가 아니라 값을 그대로 돌려준다.

표 3. 오류 경우와 메시지 예.

경우	입력 예	메시지 예
없는 단위	<code>convert(1, "km", "xx")</code>	unknown unit 'xx' (known: mm, cm, m, km, ...)
종류 불일치	<code>convert(1, "km", "kg")</code>	cannot convert length to mass
0 K 미만	<code>convert(-300, "c", "f")</code>	-300 c is below absolute zero

4 구현 계획

파일은 둘이고 새 의존성은 없다. `convert.py`는 표준 라이브러리 `import`도 하지 않는다. 표 4가 파일별 변경이다.

표 4. 바뀌는 파일.

파일	변경	이유
<code>convert.py</code> (새 파일)	LENGTH, MASS, TEMP, <code>convert()</code>	3.1-3.4절
<code>test_convert.py</code> (새 파일)	<code>unittest</code> 시험 4묶음(5절 표 5)	완료 기준 확인

작업 순서는 다음과 같다.

- ① 시험 먼저. 정의값과 기준점 시험을 먼저 적어 실패를 확인한다.
- ② 표와 함수. 표 2의 데이터와 `convert()`를 적어 ①을 통과시킨다.
- ③ 왕복·오류 시험. 나머지 시험을 추가하고 `python -m unittest`로 전부 돌린다.

5 평가 방법

부동소수점 계산은 정확한 십진수 결과를 보장하지 않는다. 설계한 식을 미리 계산해 보니 표 6처럼 마지막 자리에서 어긋났다[D3]. 그래서 시험은 같은 값 비교 대신 `math.isclose(rel_tol=1e-12, abs_tol=1e-12)`로 비교한다[R3]. 기대값이 0인 경우($32\text{ }^{\circ}\text{F} \rightarrow ^{\circ}\text{C}$) 상대 오차가 정의되지 않으므로 절대 오차 $1\text{e-}12$ 도 함께 둔다. 표 5는 시험 묶음과 완료 기준이다.

표 5. 시험 묶음과 완료 기준. 왕복 쌍 수는 길이 8×8 + 무게 6×6 + 온도 $3 \times 3 = 109$ 이다.

시험 묶음	내용	목표
정의값	1 in = 0.0254 m, 1 ft, 1 yd, 1 mi, 1 km, 1 lb, 1 oz = 1 lb ÷ 16, 1 t, 1 mg	9 / 9 통과
온도 기준점	-40 C = -40 F, 100 C = 212 F, 0 C = 273.15 K, 32 F = 0 C, 0 K = -459.67 F	5 / 5 통과
왕복	같은 종류 모든 쌍 $A \rightarrow B \rightarrow A$, 값 25(온도도 세 눈금 모두 0 K 이상)	109 / 109 통과
오류와 이름 규칙	표 3의 3종이 <code>ValueError</code> , KM → MI가 km → mi와 같음	4 / 4 통과
의존성	<code>convert.py</code> 의 <code>import</code> 문	0개

표 6. 설계한 식의 실제 계산값. Python 3.11.16에서 측정. [D3]

식	계산값	정확한 값	상대 오차
$(100 + 273.15) \times 1.8 - 459.67$	211.99999999999994	212	2.7e-16
$(-40 + 273.15) \times 1.8 - 459.67$	-40.000000000000006	-40	1.5e-15
$0.0254 \div 0.01$	2.54	2.54	0
$(32 + 459.67) \div 1.8 - 273.15$	0.0	0	0

6 위험과 대응

표 7은 M1에서 생길 수 있는 문제와 그것이 어디서 드러나는지다.

표 7. 위험과 알아채는 방법.

위험	알아채는 방법	대응
계수 오차	정의값 시험이 기대값을 따로 적어 두므로 실패한다.	계수는 표 2 그대로, 시험 기대값은 프로포절 표 3에서 따로 옮겨 적는다.
오차 누적이 1e-12를 넘음	왕복 시험 실패. 표 6의 최대 오차는 1.5e-15로 여유가 약 670배다.	넘으면 원인 식을 결과 문서에 적고 허용 오차를 승인받아 조정한다.
0 K 경계에서 반올림으로 음수	0 K = -459.67 F 시험과 -273.15 C 입력이 오류로 빠지면 드러난다.	표 6에서 경계값은 정확히 0.0으로 계산되었다. 어긋나면 결과 문서에 보고한다.

참고 출처

- [R1] NIST. NIST Guide to the SI, Appendix B.8: Factors for Units Listed Alphabetically (SP 811).
<https://www.nist.gov/pml/special-publication-811/nist-guide-si-appendix-b-conversion-factors/nist-guide-si-appendix-b8> (2026.10.09 확인)
- [R2] Wikipedia. International yard and pound.
https://en.wikipedia.org/wiki/International_yard_and_pound (2차 출처, 1959년 협정 원문은 확인 안 함)
- [R3] Python Software Foundation. math.isclose.
<https://docs.python.org/3/library/math.html#math.isclose>
- [D1] 단위 변환 CLI 프로포절 (docs/reports/unit-converter-cli/00-proposal.pdf), 2026.10.09 승인.
- [D2] 저장소 초기 커밋 254d145 “init” (파일 없음).
- [D3] 설계한 식의 계산 확인, Python 3.11.16, 2026.10.09.